Finite State Machine for English Plurals

Generating regular plural noun forms

This finite state transducer scans each noun from left to right and keeps a state for its relevant ending. Its final state selects the regular plural operation: change consonant + y to ies, add es after a sibilant ending, or add s otherwise.

Question

Implement a finite state machine for morphological parsing that generates plural forms of English nouns using Python.

Code

```
VOWELS = set("aeiou")
SIBILANT_STATES = {"S", "XZ", "CH", "SH"}


def transition(state, char):
    """Return the state that summarizes the ending seen so far."""
    if char == "y":
        return "Y_VOWEL" if state == "VOWEL" else "Y_CONSONANT"
    if char == "h" and state == "C":
        return "CH"
    if char == "h" and state == "S":
        return "SH"
    if char in VOWELS:
        return "VOWEL"
    if char == "c":
        return "C"
    if char == "s":
        return "S"
    if char in "xz":
        return "XZ"
    return "OTHER"


def pluralize(noun):
    if not noun:
        raise ValueError("noun must not be empty")
    state = "OTHER"
    for char in noun.lower():
        state = transition(state, char)
    if state == "Y_CONSONANT":
        return noun[:-1] + "ies"
    if state in SIBILANT_STATES:
        return noun + "es"
    return noun + "s"


words = ["cat", "bus", "box", "church", "baby", "key"]
print("Singular -> plural (regular English rules)")
for word in words:
    print(f"{word:8} -> {pluralize(word)}")
```

Output

```
Singular -> plural (regular English rules)
cat      -> cats
bus      -> buses
box      -> boxes
church   -> churches
baby     -> babies
key      -> keys
```

Output screenshot

